

Линейное программирование

Решение симплекс-методом

Трикула Артем Константинович

Поток: МетОпт 1.3

Вариант 11

$$Z = x_1 + 2x_2 + 4x_3 + x_4 \rightarrow \max$$

$$\begin{cases} x_1 + x_2 + x_3 \leq 10 \\ x_2 + 2x_3 + x_4 = 6 \\ x_1 + x_4 \geq 2 \end{cases}$$

$$] x_1, x_2, x_3, x_4 \geq 0$$

$$\begin{cases} x_1 + x_2 + x_3 + x_5 = 10 \\ x_2 + 2x_3 + x_4 = 6 \\ x_1 + x_4 - x_6 = 2 \\ x_{1...6} \geq 0 \end{cases}$$

Добавим x_7 и x_8 , так как x_5 уже может быть базисной

$$W_{add} = x_7 + x_8 \rightarrow \min$$

$$\begin{cases} x_1 + x_2 + x_3 + x_5 = 10 \\ x_2 + 2x_3 + x_4 + x_7 = 6 \\ x_1 + x_4 - x_6 + x_8 = 2 \\ x_{1...8} \geq 0 \end{cases}$$

$$x_7 = 6 - x_4 - 2x_3 - x_2$$

$$x_8 = 2 + x_6 - x_4 - x_1$$

$$W_{add} = 8 - x_1 - x_2 - 2x_3 - 2x_4 + x_6$$

Вспомогательная задача

	x_1	x_2	x_3	x_4	x_5	θ
x_5	1	1	1	0	0	10
x_2	0	1	2	1	0	6
x_8	1	0	0	1	-1	2
	-1	-1	-2	-2	1	-8

$10/1 = 10$
 $6/2 = 3 \leftarrow$
 $\Downarrow$
 a_{23}

↑

	x_1	x_2	x_3	x_4	x_5	θ
x_5	1	$1/2$	$-1/2$	$-1/2$	0	4
x_3	0	$1/2$	$1/2$	$1/2$	0	3
x_8	1	0	0	1	-1	2
	-1	0	-1	-1	1	-2

	x_1	x_2	x_3	x_4	θ
x_5	1	$1/2$	$-1/2$	0	4
x_3	0	$1/2$	$1/2$	0	3
x_8	1	0	1	-1	2
	-1	0	-1	1	-2

$4/1 = 4$
 $2/1 = 2$

$\Rightarrow a_{81}$

↑

	x_8	x_2	x_3	x_4	θ
x_5	-1	$1/2$	$-3/2$	1	5
x_3	0	$1/2$	$1/2$	0	3
x_1	1	0	1	-1	2
	1	0	0	0	0

Основная задача

	x_2	x_4	x_6	b
x_5	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	0	1	-1	2
	0	0	0	0

$$\begin{cases} x_5 + \frac{1}{2}x_2 - \frac{3}{2}x_4 + x_6 = 5 \\ x_3 + \frac{1}{2}x_2 + \frac{1}{2}x_4 = 3 \\ x_1 + x_4 - x_6 = 2 \end{cases}$$

$$] W = -Z = -x_1 - 2x_2 - 4x_3 - x_4 \rightarrow \min$$

выразим базисные переменные:

$$W = -(2 - x_4 + x_6) - 2x_2 - 4\left(3 - \frac{1}{2}x_2 - \frac{1}{2}x_4\right) - x_4 = -14 + 2x_4 - x_6$$

	x_2	x_4	x_6	b
x_5	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	0	1	-1	2
	0	2	-1	14



	x_2	x_4	x_5	b
x_6	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	$1/2$	$-1/2$	1	7
	$1/2$	$1/2$	1	19

Ответ и проверка

$$x_2 = x_4 = x_5 = 0$$

$$x_6 = 5$$

$$x_3 = 3$$

$$x_1 = 7$$

$$x^* = (7, 0, 3, 0)$$

Проверка:

$$Z(x^*) = 1 \cdot 7 + 4 \cdot 3 = 19$$

$$\begin{cases} 7 + 0 + 3 \leq 10 \\ 0 + 2 \cdot 3 + 0 = 6 \\ 7 + 0 \geq 2 \end{cases}$$

Проверка в Excel Solver

C	1	2	4	1		
X	7	0	3	0	19	
	1	1	1	0	10	10
	0	1	2	1	6	6
	1	0	0	1	7	2

Excel: $x_1 = 7$, $x_2 = 0$, $x_3 = 3$, $x_4 = 0$; $Z = 19$. Значения левых частей ограничений: $10 \leq 10$, $6 = 6$, $7 \geq 2$.
Результат совпадает с ручным решением.

Рефлексивный вывод

Работа показалась интересной и полезной. Важно не просто прослушать лекции и практическое занятие, но и самому вручную использовать симплекс-метод. Было трудоёмко разбираться с вычислением отдельных коэффициентов при перемещении переменных из свободных в базисные и вычислять множество элементов таблицы. Основные сложности были связаны именно с этим. На последнем шаге я забыл поменять местами базисную и свободную переменные, вследствие чего пришлось некоторое время искать эту ошибку. В целом других трудностей не было, не считая большого количества несложных арифметических операций.

Программная реализация

Задача решена на Python двухфазным симплекс-методом

Порядок работы программы

1. Приведение ограничений и границ переменных к каноническому виду
2. Построение и решение вспомогательной задачи
3. Удаление искусственных переменных и переход к основной цели
4. Решение основной задачи и восстановление исходных переменных
5. Независимое сравнение результата с SciPy

Реализация

Выбор разрешающего элемента, пересчёт таблиц, обработка статусов и переход между фазами реализованы самостоятельно

NumPy используется для массивов и матричных вычислений

Rich оформляет таблицы в консоли, SciPy проверяет результат через HiGHS

Структура исходного кода

models.py - постановка задачи, каноническая форма, снимок и результат

simplex.py - сокращённая таблица и двухфазный симплекс-метод

console_tables.py - оформление истории таблиц

variant11.py - последовательный запуск варианта 11 и проверка SciPy

В программе s1 соответствует ручной переменной x5, s3 - x6,

a2 - x7, a3 - x8. Исходные переменные x1, x2, x3, x4 совпадают

Правило most-negative воспроизводит последовательность ручных обменов

Запуск из корня проекта

```
poetry run python lab01/src/variant11.py
```

Получено 3 обмена: x3 вместо a2; x1 вместо a3; s3 вместо s1

Оптимальная точка: $x^* = (7, 0, 3, 0)$

Значение целевой функции: $Z = 19$

Результаты ручного решения, Excel Solver и SciPy совпадают

Вывод программы

s - дополнительная переменная, a - искусственная; Цель: коэффициенты и $-Q$.

Фаза I. Вспомогательная задача

01. Начальная таблица вспомогательной задачи

Базис	x_1	x_2	x_3	x_4	s_3	$b / -Q$
s_1	1	1	1	0	0	10
a_2	0	1	2	1	0	6
a_3	1	0	0	1	-1	2
Цель	-1	-1	-2	-2	1	-8

02. Входит x_3 , выходит a_2 ; элемент 2

Базис	x_1	x_2	a_2	x_4	s_3	$b / -Q$
s_1	1	1/2	-1/2	-1/2	0	7
x_3	0	1/2	1/2	1/2	0	3
a_3	1	0	0	1	-1	2
Цель	-1	0	1	-1	1	-2

03. Удалён столбец искусственной переменной a_2

Базис	x_1	x_2	x_4	s_3	$b / -Q$
s_1	1	1/2	-1/2	0	7
x_3	0	1/2	1/2	0	3
a_3	1	0	1	-1	2
Цель	-1	0	-1	1	-2

04. Входит x_1 , выходит a_3 ; элемент 1

Базис	a_3	x_2	x_4	s_3	$b / -Q$
s_1	-1	1/2	-3/2	1	5
x_3	0	1/2	1/2	0	3
x_1	1	0	1	-1	2
Цель	1	0	0	0	0

Вывод программы

05. Удалён столбец искусственной переменной a_3

Базис	x_2	x_4	s_3	$b / -Q$
s_1	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	0	1	-1	2
Цель	0	0	0	0

Фаза II. Основная задача

06. Возвращение к основной целевой функции

Базис	x_2	x_4	s_3	$b / -Q$
s_1	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	0	1	-1	2
Цель	0	2	-1	14

07. Входит s_3 , выходит s_1 ; элемент 1

Базис	x_2	x_4	s_1	$b / -Q$
s_3	$1/2$	$-3/2$	1	5
x_3	$1/2$	$1/2$	0	3
x_1	$1/2$	$-1/2$	1	7
Цель	$1/2$	$1/2$	1	19

Статус: optimal. Итераций: 3

$x^* = [7. \ 0. \ 3. \ 0.]$

Целевая функция = 19

SciPy $x^* = [7. \ 0. \ 3. \ 0.]$

SciPy: целевая функция = 19

Проверка SciPy: совпадает

Листинг: variant11.py

```
1 import numpy as np
2 from scipy.optimize import linprog
3
4 from console_tables import ConsoleTablePrinter
5 from models import LinearProgram
6 from simplex import TwoPhaseSimplex
7
8
9 problem = LinearProgram(
10     c=np.array([1, 2, 4, 1], dtype=float),
11     A=np.array([[1, 1, 1, 0], [0, 1, 2, 1], [1, 0, 0, 1]], dtype=float),
12     b=np.array([10, 6, 2], dtype=float),
13     signs=["<=", "=", ">="],
14     sense="max",
15 )
16
17 solver = TwoPhaseSimplex(pivot_rule="most-negative")
18 solution = solver.solve(problem)
19
20 print("s - дополнительная переменная, а - искусственная; Цель: коэффициенты и -Q.")
21 ConsoleTablePrinter().print_steps(solver.history)
22
23 print(f"\nСтатус: {solution.status}. Итераций: {solution.iterations}")
24 if solution.status == "optimal" and solution.x is not None and solution.value is not
    None:
25     print("x* =", np.round(solution.x, 8))
26     print(f"Целевая функция = {solution.value:.8g}")
27
28 reference = linprog(
29     -problem.c,
```

Листинг: variant11.py

```
30     A_ub=[problem.A[0], -problem.A[2]],
31     b_ub=[problem.b[0], -problem.b[2]],
32     A_eq=[problem.A[1]],
33     b_eq=[problem.b[1]],
34     bounds=problem.bounds,
35     method="highs",
36 )
37
38 if (
39     reference.success
40     and solution.status == "optimal"
41     and solution.x is not None
42     and solution.value is not None
43 ):
44     lhs = problem.A @ solution.x
45     feasible = (
46         lhs[0] <= problem.b[0] + 1e-7
47         and np.isclose(lhs[1], problem.b[1], atol=1e-7, rtol=0)
48         and lhs[2] >= problem.b[2] - 1e-7
49         and np.all(solution.x >= -1e-7)
50     )
51     matches = feasible and np.isclose(solution.value, -reference.fun)
52
53     print("\nSciPy x* =", np.round(reference.x, 8))
54     print(f"SciPy: целевая функция = {-reference.fun:.8g}")
55     print("Проверка SciPy:", "совпадает" if matches else "НЕ совпадает")
56 else:
57     print("\nПроверка SciPy: оптимальное решение не получено.")
58     print(reference.message)
```

Листинг: models.py

```
1 from dataclasses import dataclass
2 from typing import Literal
3
4 import numpy as np
5
6
7 @dataclass
8 class LinearProgram:
9     """Исходная задача линейного программирования
10
11     :param c: Коэффициенты целевой функции
12     :param A: Матрица ограничений
13     :param b: Правые части ограничений
14     :param signs: Знаки ограничений: <=, >= или =
15     :param sense: Направление оптимизации: min или max
16     :param bounds: Границы переменных; по умолчанию (0, None)
17     """
18
19     c: np.ndarray
20     A: np.ndarray
21     b: np.ndarray
22     signs: list[str]
23     sense: Literal["min", "max"] = "min"
24     bounds: list[tuple[float | None, float | None]] | None = None
25
26     def __post_init__(self):
27         """Преобразует входные данные в массивы и проверяет их согласованность
28
29         :return: None
30         """
31         self.c = np.asarray(self.c, dtype=float).reshape(-1)
32         self.b = np.asarray(self.b, dtype=float).reshape(-1)
33         self.A = np.asarray(self.A, dtype=float).reshape(len(self.b), len(self.c))
34
35         if len(self.signs) != len(self.b) or any(s not in {"<=", ">=", "="} for s in
36 self.signs):
37             raise ValueError("Для каждого ограничения укажите знак <=, >= или =.")
38         if self.sense not in {"min", "max"}:
39             raise ValueError("Направление оптимизации: min или max.")
40         if self.bounds is None:
41             self.bounds = [(0, None)] * len(self.c)
42         elif len(self.bounds) != len(self.c):
43             raise ValueError("Число пар bounds должно совпадать с числом
44 переменных.")
```

Листинг: models.py

```

45 @dataclass
46 class CanonicalForm:
47     """Каноническая задача минимизации с начальным базисом
48
49     :param A: Матрица равенств
50     :param b: Правые части равенств
51     :param c: Коэффициенты преобразованной цели
52     :param constant: Свободный член цели
53     :param basis: Индексы базисных переменных
54     :param artificial: Индексы искусственных переменных
55     :param names: Имена переменных
56     :param shift: Сдвиг исходных переменных
57     :param transform: Матрица замены  $x = \text{shift} + \text{transform} @ y$ 
58     """
59
60     A: np.ndarray
61     b: np.ndarray
62     c: np.ndarray
63     constant: float
64     basis: list[int]
65     artificial: set[int]
66     names: list[str]
67     shift: np.ndarray
68     transform: np.ndarray
69
70     @classmethod
71     def from_problem(cls, problem: LinearProgram):
72         """Приводит исходную задачу к каноническому виду
73
74         :param problem: Исходная постановка задачи
75         :return: Каноническая форма с базисом вспомогательной задачи
76         """
77         n = len(problem.c)
78         shift = np.zeros(n)
79         columns, names, upper_constraints = [], [], []
80
81         for j, (lower, upper) in enumerate(problem.bounds):
82             unit = np.eye(n)[j, j]
83             if lower is not None:
84                 shift[j] = lower
85                 columns.append(unit)
86                 names.append(f"x{j + 1}" if lower == 0 else f"y{j + 1}")
87                 if upper is not None:
88                     upper_constraints.append((len(columns) - 1, upper - lower))
89             elif upper is not None:
90                 shift[j] = upper

```

Листинг: models.py

```

91         columns.append(-unit)
92         names.append(f"y{j + 1}")
93     else:
94         columns.extend([unit, -unit])
95         names.extend([f"x{j + 1}+", f"x{j + 1}-"])
96
97     transform = np.column_stack(columns)
98     A = problem.A @ transform
99     b = problem.b - problem.A @ shift
100     signs = list(problem.signs)
101
102     for index, upper in upper_constraints:
103         row = np.zeros(transform.shape[1])
104         row[index] = 1
105         A = np.vstack([A, row])
106         b = np.append(b, upper)
107         signs.append("<=")
108
109     # Неотрицательная правая часть нужна для допустимого начального базиса.
110     for i in range(len(b)):
111         if b[i] < 0:
112             A[i] *= -1
113             b[i] *= -1
114             signs[i] = {"<=": ">=", ">=": "<=", "=": "="}[signs[i]]
115
116     direction = 1 if problem.sense == "min" else -1
117     c = direction * (problem.c @ transform)
118     constant = float(direction * (problem.c @ shift))
119     basis, artificial = [], set()
120
121     def add_column(i, value, artificial_column=False):
122         """Добавляет дополнительную или искусственную переменную
123
124         :param i: Индекс ограничения
125         :param value: Коэффициент нового столбца в строке i
126         :param artificial_column: Признак искусственной переменной
127         :return: Индекс добавленного столбца
128         """
129         nonlocal A, c
130         column = np.zeros(len(b))
131         column[i] = value
132         A = np.column_stack([A, column])
133         c = np.append(c, 0)
134         index = A.shape[1] - 1
135         names.append(("a" if artificial_column else "s") + str(i + 1))
136         if artificial_column:

```

Листинг: models.py

```

137         artificial.add(index)
138     return index
139
140     for i, sign in enumerate(signs):
141         if sign == "<=":
142             basis.append(add_column(i, 1))
143         else:
144             if sign == ">=":
145                 add_column(i, -1)
146             basis.append(add_column(i, 1, artificial_column=True))
147
148     return cls(A, b, c, constant, basis, artificial, names, shift, transform)
149
150
151 @dataclass
152 class TableSnapshot:
153     """Снимок симплекс-таблицы для вывода шагов
154
155     :param phase: Фаза I или II
156     :param action: Описание шага
157     :param basis: Имена базисных переменных
158     :param nonbasis: Имена свободных переменных
159     :param values: Коэффициенты таблицы, правые части и строка цели
160     """
161     phase: str
162     action: str
163     basis: list[str]
164     nonbasis: list[str]
165     values: np.ndarray
166
167
168 @dataclass
169 class Solution:
170     """Результат решения исходной задачи
171
172     :param status: optimal, infeasible, unbounded или iteration-limit
173     :param x: Оптимальная точка; None, если оптимум не найден
174     :param value: Значение цели в оптимуме или None
175     :param iterations: Число выполненных обменов
176     """
177     status: Literal["optimal", "infeasible", "unbounded", "iteration-limit"]
178     x: np.ndarray | None = None
179     value: float | None = None
180     iterations: int = 0

```

Листинг: simplex.py

```
1 import numpy as np
2
3 from models import CanonicalForm, LinearProgram, Solution, TableSnapshot
4
5
6 class SimplexTableau:
7     """Сокращённая симплекс-таблица со строкой цели (r, -Q)"""
8
9     def __init__(self, form: CanonicalForm, tolerance: float):
10         """Создаёт начальную симплекс-таблицу
11
12         :param form: Каноническая постановка задачи
13         :param tolerance: Числовой допуск
14         :return: None
15         """
16         self.basis = list(form.basis)
17         self.nonbasis = [j for j in range(len(form.c)) if j not in self.basis]
18         self.tolerance = tolerance
19         self.values = np.zeros((len(form.b) + 1, len(self.nonbasis) + 1))
20
21         # При построении выбранные базисные столбцы уже образуют единичную матрицу
22         self.values[:-1, :-1] = form.A[:, self.nonbasis]
23         self.values[:-1, -1] = form.b
24
25     def set_objective(self, costs, constant=0.0):
26         """Пересчитывает строку цели для текущего базиса
27
28         :param costs: Коэффициенты минимизируемой цели
29         :param constant: Свободный член цели
30         :return: None
31         """
32         basic_costs = costs[self.basis]
33         self.values[-1, :-1] = costs[self.nonbasis] - basic_costs @ self.values[:-1,
34 :-1]
35         self.values[-1, -1] = -(constant + basic_costs @ self.values[:-1, -1])
36
37     def pivot(self, row, column):
38         """Обменивает базисную и свободную переменные
39
40         :param row: Индекс разрешающей строки
41         :param column: Индекс разрешающего столбца
42         :return: None
43         """
44         old = self.values.copy()
```

Листинг: simplex.py

```
44     alpha = old[row, column]
45
46     # Правило прямоугольника для всех обычных элементов, включая строку цели и b
47     self.values = old - np.outer(old[:, column], old[row, :]) / alpha
48     self.values[row, :] = old[row, :] / alpha
49     self.values[:, column] = -old[:, column] / alpha
50     self.values[row, column] = 1 / alpha
51
52     self.basis[row], self.nonbasis[column] = self.nonbasis[column],
self.basis[row]
53
54     def drop_column(self, column):
55         """Удаляет столбец свободной переменной
56
57         :param column: Позиция столбца в таблице
58         :return: None
59         """
60         self.values = np.delete(self.values, column, axis=1)
61         self.nonbasis.pop(column)
62
63     def drop_row(self, row):
64         """Удаляет избыточное ограничение
65
66         :param row: Позиция строки в таблице
67         :return: None
68         """
69         self.values = np.delete(self.values, row, axis=0)
70         self.basis.pop(row)
71
72
73     class TwoPhaseSimplex:
74         """Двухфазный симплекс-метод с историей таблиц"""
75
76         form: CanonicalForm
77         table: SimplexTableau
78
79         def __init__(self, tolerance=1e-9, max_iterations=1000, pivot_rule="bland"):
80             """Настраивает точность и правило выбора переменной
81
82             :param tolerance: Допуск для сравнений
83             :param max_iterations: Общий лимит обменов
84             :param pivot_rule: bland или most-negative
85             :return: None
86             """
```

Листинг: simplex.py

```
87         self.tolerance = tolerance
88         self.max_iterations = max_iterations
89         if pivot_rule not in {"bland", "most-negative"}:
90             raise ValueError("Правило выбора: bland или most-negative.")
91         self.pivot_rule = pivot_rule
92         self.history: list[TableSnapshot] = []
93         self.iterations = 0
94
95     def _remember(self, phase, action):
96         """Сохраняет снимок текущей таблицы
97
98         :param phase: Фаза I или II
99         :param action: Описание выполненного шага
100         :return: None
101         """
102         self.history.append(TableSnapshot(
103             phase, action,
104             [self.form.names[j] for j in self.table.basis],
105             [self.form.names[j] for j in self.table.nonbasis],
106             self.table.values.copy(),
107         ))
108
109     def _optimize(self, phase):
110         """Выполняет симплекс-итерации текущей фазы
111
112         :param phase: Фаза I или II
113         :return: Статус optimal, unbounded или iteration-limit
114         """
115         while True:
116             candidates = np.flatnonzero(self.table.values[-1, :-1] < -self.tolerance)
117             if not len(candidates):
118                 return "optimal"
119             if self.iterations >= self.max_iterations:
120                 return "iteration-limit"
121
122             if self.pivot_rule == "bland":
123                 # Наименьший индекс переменной, а не позиция переставленного столбца.
124                 column = min(candidates, key=lambda j: self.table.nonbasis[j])
125             else:
126                 column = min(candidates, key=lambda j: (self.table.values[-1, j],
self.table.nonbasis[j]))
127
128             eligible = np.flatnonzero(self.table.values[:-1, column] >
self.tolerance)
```

Листинг: simplex.py

```

129         if not len(eligible):
130             return "unbounded"
131
132         ratios = self.table.values[eligible, -1] / self.table.values[eligible,
column]
133         minimum = ratios.min()
134         tied = eligible[np.abs(ratios - minimum) <= self.tolerance]
135         row = min(tied, key=lambda i: self.table.basis[i])
136
137         entering = self.form.names[self.table.nonbasis[column]]
138         leaving_id = self.table.basis[row]
139         leaving = self.form.names[leaving_id]
140         alpha = self.table.values[row, column]
141
142         self.table.pivot(row, column)
143         self.iterations += 1
144         self._remember(phase, f"Входит {entering}, выходит {leaving}; элемент
{alpha:g}")
145
146         if leaving_id in self.form.artificial:
147             self.table.drop_column(column)
148             self._remember(phase, f"Удалён столбец искусственной переменной
{leaving}")
149
150     def solve(self, problem: LinearProgram) -> Solution:
151         """Решает задачу и восстанавливает исходные переменные
152
153         :param problem: Исходная постановка задачи
154         :return: Статус, оптимальная точка, значение цели и число обменов
155         """
156         self.history = []
157         self.iterations = 0
158         self.form = CanonicalForm.from_problem(problem)
159         self.table = SimplexTableau(self.form, self.tolerance)
160
161         auxiliary_costs = np.zeros(len(self.form.c))
162         auxiliary_costs[list(self.form.artificial)] = 1
163         self.table.set_objective(auxiliary_costs)
164         self._remember("I", "Начальная таблица вспомогательной задачи")
165
166         status = self._optimize("I")
167         if status == "iteration-limit":
168             return Solution(status, iterations=self.iterations)
169         if status == "unbounded":

```

Листинг: simplex.py

```

170         # Сумма неотрицательных искусственных переменных не может уходить в минус
        бесконечность
171         raise ArithmeticError("Численная ошибка в фазе I; проверьте масштаб
коэффициентов.")
172         if -self.table.values[-1, -1] > self.tolerance:
173             return Solution("infeasible", iterations=self.iterations)
174
175         # Нулевая искусственная переменная может остаться базисной при вырождении
176         for row in range(len(self.table.basis) - 1, -1, -1):
177             if self.table.basis[row] not in self.form.artificial:
178                 continue
179             candidates = [
180                 j for j, index in enumerate(self.table.nonbasis)
181                 if index not in self.form.artificial
182                 and abs(self.table.values[row, j]) > self.tolerance
183             ]
184             if candidates:
185                 if self.iterations >= self.max_iterations:
186                     return Solution("iteration-limit", iterations=self.iterations)
187                 column = min(candidates, key=lambda j: self.table.nonbasis[j])
188                 self.table.pivot(row, column)
189                 self.iterations += 1
190                 self.table.drop_column(column)
191                 self._remember("I", "Выведена нулевая искусственная переменная")
192             else:
193                 self.table.drop_row(row)
194                 self._remember("I", "Удалено избыточное равенство  $0 = 0$ ")
195
196         for column in range(len(self.table.nonbasis) - 1, -1, -1):
197             if self.table.nonbasis[column] in self.form.artificial:
198                 self.table.drop_column(column)
199
200         self.table.set_objective(self.form.c, self.form.constant)
201         self._remember("II", "Возвращение к основной целевой функции")
202         status = self._optimize("II")
203         if status != "optimal":
204             return Solution(status, iterations=self.iterations)
205
206         canonical_x = np.zeros(len(self.form.c))
207         canonical_x[self.table.basis] = self.table.values[: -1, -1]
208         y = canonical_x[:self.form.transform.shape[1]]
209         x = self.form.shift + self.form.transform @ y
210
211         return Solution("optimal", x, float(problem.c @ x), self.iterations)

```

Листинг: console_tables.py

```
1 from fractions import Fraction
2 from math import isclose
3 from typing import TYPE_CHECKING, Iterable
4
5 from rich import box
6 from rich.console import Console
7 from rich.table import Table
8 from rich.text import Text
9
10 if TYPE_CHECKING:
11     from models import TableSnapshot
12
13
14 class ConsoleTablePrinter:
15     """Вывод симплекс-таблиц в консоль через Rich"""
16
17     def __init__(self, console: Console | None = None):
18         """Настраивает консоль для вывода таблиц
19
20         :param console: Консоль Rich; по умолчанию создаётся новая
21         :return: None
22         """
23         self.console = console if console is not None else Console()
24
25     @staticmethod
26     def format_number(value: float) -> str:
27         """Форматирует число как дробь или десятичную запись
28
29         :param value: Число для отображения
30         :return: Строковое представление числа
31         """
32         value = float(value)
33         fraction = Fraction(value).limit_denominator(10000)
34
35         if isclose(value, float(fraction), rel_tol=1e-10, abs_tol=1e-12):
36             return str(fraction)
37
38         return f"{value:.8g}"
39
40     def print_steps(self, history: Iterable["TableSnapshot"]):
41         """Выводит историю таблиц с заголовками фаз
42
```

Листинг: console_tables.py

```

43         :param history: Последовательность снимков таблиц
44         :return: None
45         """
46         current_phase = None
47
48         for index, step in enumerate(history, start=1):
49             if step.phase != current_phase:
50                 title = "Вспомогательная задача" if step.phase == "I" else "Основная
задача"
51                 self.console.print()
52                 self.console.print(Text(f"Фаза {step.phase}. {title}", style="bold
53                 cyan"))
54                 current_phase = step.phase
55
56                 self.console.print()
57                 self.console.print(Text(f"{index:02d}. {step.action}", style="bold"))
58
59                 table = Table(
60                     box=box.ROUNDED,
61                     header_style="bold cyan",
62                     border_style="dim",
63                     padding=(0, 1),
64                     expand=False,
65                 )
66
67                 table.add_column("Базис", justify="left", no_wrap=True)
68                 for name in step.nonbasis:
69                     table.add_column(name, justify="right", no_wrap=True)
70                     table.add_column("b / -Q", justify="right", no_wrap=True, style="bold")
71
72                 for name, values in zip(step.basis, step.values[:-1]):
73                     table.add_row(name, *(self.format_number(v) for v in values))
74
75                 if step.basis:
76                     table.add_section()
77                     table.add_row(
78                         "Цель", *(self.format_number(v) for v in step.values[-1]),
79                         style="bold cyan"
80                     )
81
82                 self.console.print(table)

```